FATIMA JINNAH WOMEN UNIVERSITY

Rawalpindi

DevOps – Lab 05

Building and Packaging Applications with Maven, Gradle, npm and Docker

Name	Mahnoor Sajjad
Roll No.	2023-BCS-054 (Section A)
Program	BS Computer Science
Course	DevOps
Lab	Lab 05 (Build Tools & Docker Images)
GitHub Username	mahnoor-cs6767
Repository	https://github.com/WaqasSaleem97/Intellij-Idea-docker

Objective

This lab shows how to build, test, package and run applications with different build tools: Maven and Gradle for Java, npm for Node.js, and Docker for container images. All work is done with IntelliJ IDEA running inside a Docker container. You build two Java applications and add a dependency to each, then test and package a Node.js application as an npm package and as a Docker image, and finally bundle the API of a React and Node.js project with webpack.

Tools and technologies

- Docker and Docker Compose (running IntelliJ IDEA in a container)
- Maven 3.9.12 and the Gradle wrapper for Java builds (Spring Boot sample applications)
- Node.js 22.23.4 and npm 10.9.4, with Jest for testing and webpack 4 for bundling
- Git, cloning from GitHub and GitLab

Lab overview

Part	Topic	Project	Steps
1	Environment setup and tool check	Intellij-Idea-docker	1–3
2	Build, package and run with Maven	java-maven-app	4–11
3	Build, package and run with Gradle	java-app	12–18
4	Install, test, package and run with npm	node-project	19–24
5	Build a Docker image	node-project	25–28
6	Bundle a Node.js API with webpack	react-nodejs-example	29–32

Part 1: Environment Setup (IntelliJ IDEA in Docker)

Step 1: Clone the IntelliJ-in-Docker repository

This lab uses a ready-made Docker setup that runs IntelliJ IDEA inside a container, so you can build projects without installing an IDE on your machine. Clone the repository that contains this setup.

What to do

1. Go to your home folder:

```
cd ~
```

2. Clone the repository and enter its folder:

```
git clone https://github.com/WaqasSaleem97/Intellij-Idea-docker.git
cd Intellij-Idea-docker
```

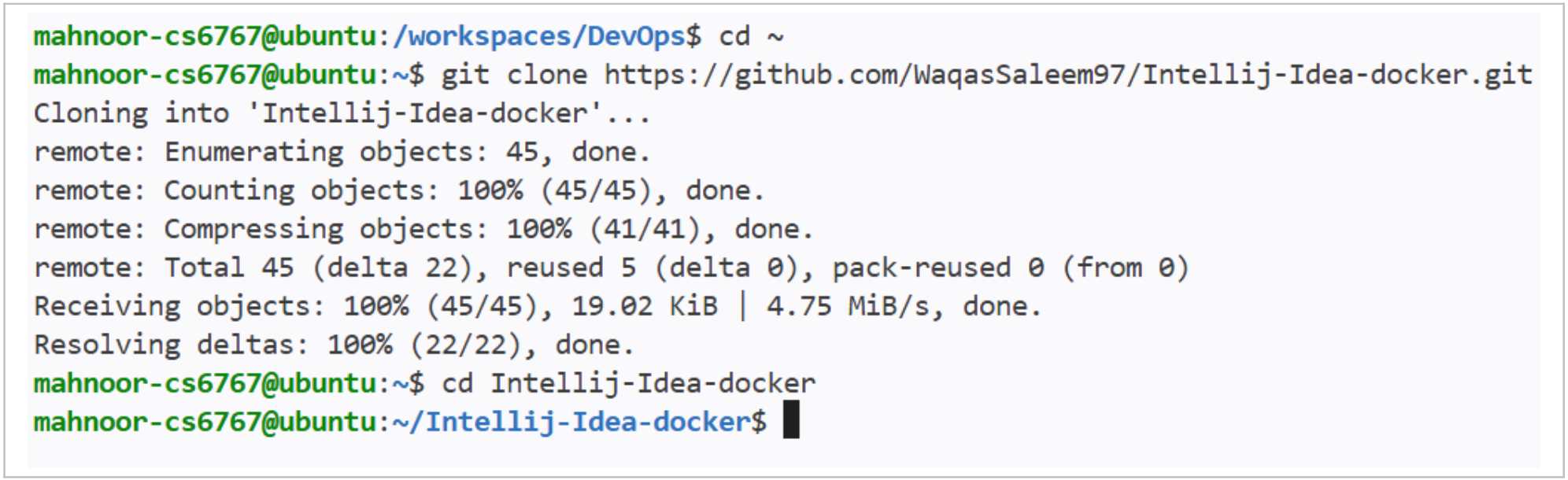


Figure 1: Cloning the Intellij-Idea-docker repository

What to look for

- All 45 objects are received (100%) and “Resolving deltas” finishes.
- The prompt changes to ~/Intellij-Idea-docker, confirming you are inside the project folder.

Step 2: Start the IntelliJ container and open the IDE

Docker Compose starts the container that hosts IntelliJ IDEA. Check that it is running, then open the IntelliJ desktop.

What to do

1. Check that the container is up:

```
docker compose ps
```

```
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker$ docker compose ps
NAME                              IMAGE                             COMMAND   SERVICE    CREATED              STATUS              PORTS
intellij-idea-docker-intellij-1   intellij-idea-docker-intellij     "/init"   intellij   About a minute ago   Up About a minute   3000/tc
p, 127.0.0.1:3001->3001/tcp
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker$
```

Figure 2: The IntelliJ container is running

2. If the desktop needs a refresh, restart the container. The IntelliJ “Welcome” screen then appears (left), with the restart shown in the terminal (right):

```
docker compose restart
```

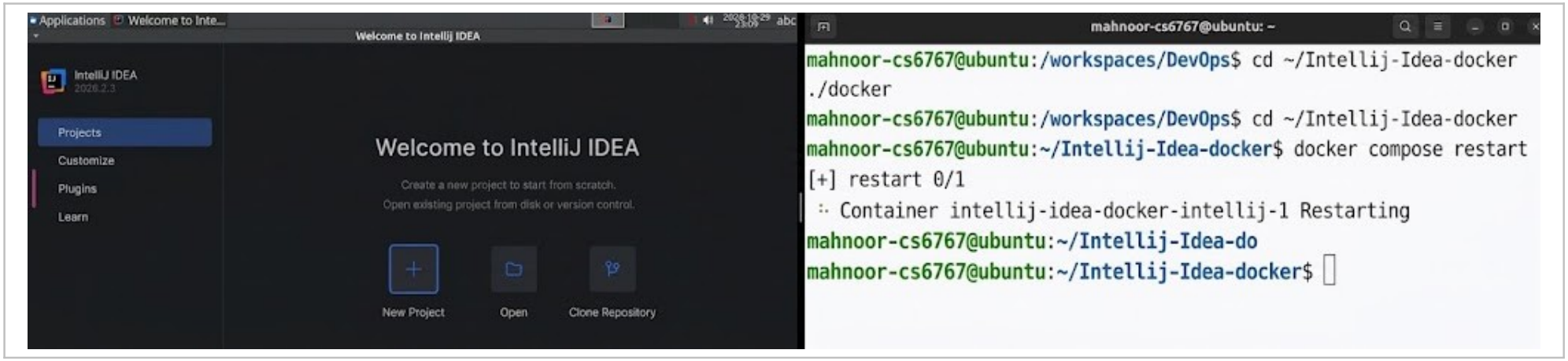


Figure 3: IntelliJ IDEA Welcome screen and container restart

What to look for

- The container intellij-idea-docker-intellij-1 shows “Up” and publishes port 3000 and 127.0.0.1:3001.
- The Welcome screen offers New Project, Open and Clone Repository, which you use in the next parts.

Step 3: Verify Docker, Maven, Node.js and npm

Before building anything, confirm that the tools used in this lab are installed. Docker and Maven are checked on your machine; Node.js and npm are checked inside the IntelliJ container.

What to do

1. Check Docker:

```
docker --version
```

```
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker/config/IdeaProjects/node-project$ docker --version
Docker version 29.9.2, build a82e528
```

Figure 4: Docker version

2. Check Maven (this also shows the Java version it uses):

```
mvn --version
```

```
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker$ mvn --version
Apache Maven 3.9.12
Maven home: /usr/share/maven
Java version: 25.0.4.1, vendor: Ubuntu, runtime: /usr/lib/jvm/java-25-openjdk-amd64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.18.54.1-microsoft-standard-wsl2", arch: "amd64", family: "unix"
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker$
```

Figure 5: Apache Maven version and Java details

3. Check Node.js and npm inside the container:

```
docker exec -it intellij-idea-docker-intellij-1 bash -c "node -v && npm -v"
```

```
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker/config/IdeaProjects/node-project$ docker exec -it intellij-idea-docker-intellij-1 bash -c "node -v && npm -v"
v22.23.4
10.9.9
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker/config/IdeaProjects/node-project$
```

Figure 6: Node.js and npm versions inside the container

What to look for

- Docker 29.9.2, Apache Maven 3.9.12 (running on Java 25), Node.js v22.23.4 and npm 10.9.4 are installed.
- Each command prints a version instead of an error.

Part 2: Build a Java Application with Maven (java-maven-app)

Step 4: Clone the java-maven-app project in IntelliJ

Use IntelliJ's built-in Git support to download the sample Maven project.

What to do

1. On the IntelliJ Welcome screen, click Clone Repository.
2. Choose Git as the version control, enter the repository URL and keep the suggested directory:

```
URL: https://gitlab.com/nanuchi/java-maven-app.git
Directory: /config/IdeaProjects/java-maven-app
```

3. Click Clone.

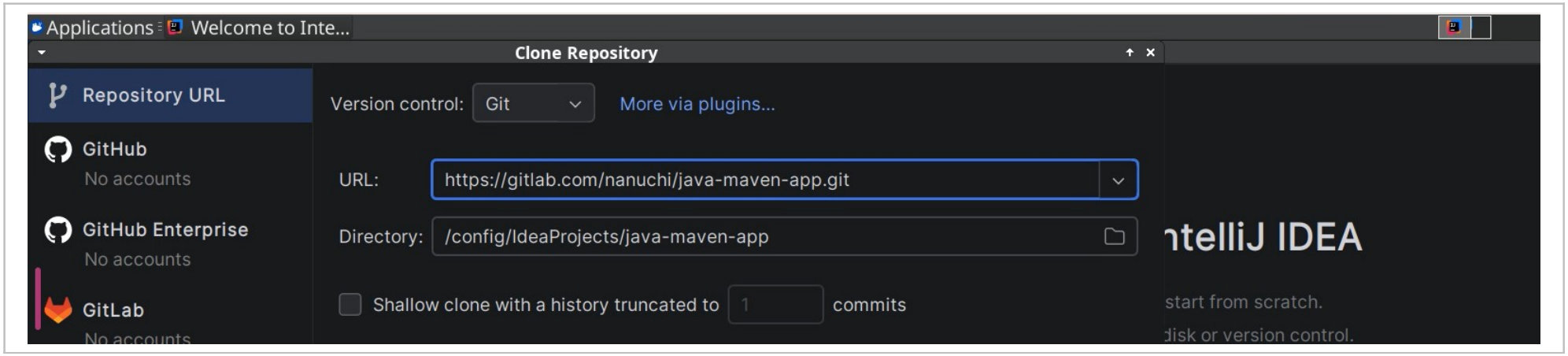


Figure 7: Clone Repository dialog for java-maven-app

What to look for

- The URL and directory match the values above.
- The GitHub and GitLab sections say "No accounts", which is fine because the repository is public.

Step 5: Build the project with Maven

Maven reads pom.xml, downloads the libraries and plugins it needs, then compiles the code and packages it into a JAR file.

What to do

1. In the project folder, run the package goal:

```
mvn package
```

The command is not visible in the screenshot; this is the standard command that produces this output.

```
Downloaded from central: https://repo.maven.apache.org/maven2/net/java/dev/jna/jna-platform/5.5.0/jna-platform-5.5.0.jar (2.7 MB at 4.2 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/eclipse/sisu/org.eclipse.sisu.plexus/0.3.4/org.eclipse.sisu.plexus-0.3.4.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-component-annotations/1.5.5/plexus-component-annotations-1.5.5.jar (4.2 kB at 6.2 k
B/s)
Downloading from central: https://repo.maven.apache.org/maven2/javax/enterprise/cdi-api/1.0/cdi-api-1.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-api/3.6.3/maven-plugin-api-3.6.3.jar (47 kB at 67 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/javax/annotation/jsr250-api/1.0/jsr250-api-1.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/eclipse/sisu/org.eclipse.sisu.plexus/0.3.4/org.eclipse.sisu.plexus-0.3.4.jar (205 kB at 289 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/eclipse/sisu/org.eclipse.sisu.inject/0.3.4/org.eclipse.sisu.inject-0.3.4.jar
Downloaded from central: https://repo.maven.apache.org/maven2/javax/enterprise/cdi-api/1.0/cdi-api-1.0.jar (45 kB at 62 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.2.1/plexus-utils-3.2.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/commons-codec/commons-codec/1.11/commons-codec-1.11.jar (335 kB at 446 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-classworlds/2.6.0/plexus-classworlds-2.6.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/javax/annotation/jsr250-api/1.0/jsr250-api-1.0.jar (5.8 kB at 7.8 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/eclipse/sisu/org.eclipse.sisu.inject/0.3.4/org.eclipse.sisu.inject-0.3.4.jar (379 kB at 475 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.2.1/plexus-utils-3.2.1.jar (262 kB at 320 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-classworlds/2.6.0/plexus-classworlds-2.6.0.jar (53 kB at 62 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/spring-core/5.2.10.RELEASE/spring-core-5.2.10.RELEASE.jar (1.4 MB at 934 kB/s)
[INFO] Replacing main artifact with repackaged archive
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  40.276 s
[INFO] Finished at: 2026-10-30T10:41:48+05:00
[INFO] ------------------------------------------------------------------------
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker/config/IdeaProjects/java-maven-app$
```

Figure 8: Maven package build finishing with BUILD SUCCESS

What to look for

- Many “Downloaded from central” lines appear because this is the first build and Maven must fetch its dependencies.
- “Replacing main artifact with repackaged archive” shows that Spring Boot turned the JAR into a runnable one.
- The build ends with BUILD SUCCESS (about 40 seconds, finished at 10:41:48).

Step 6: Add the Elasticsearch dependency and rebuild

A dependency is a library your project needs. Adding it to pom.xml makes Maven download it automatically.

What to do

1. Open pom.xml in IntelliJ and add this block inside the <dependencies> section:

```
<dependency>
    <groupId>org.elasticsearch</groupId>
    <artifactId>elasticsearch</artifactId>
    <version>7.0.0</version>
    <scope>compile</scope>
</dependency>
```

2. Rebuild so Maven downloads the new library:

```
mvn compile
```

The command is not visible in the screenshot; this is the standard command that produces this output.

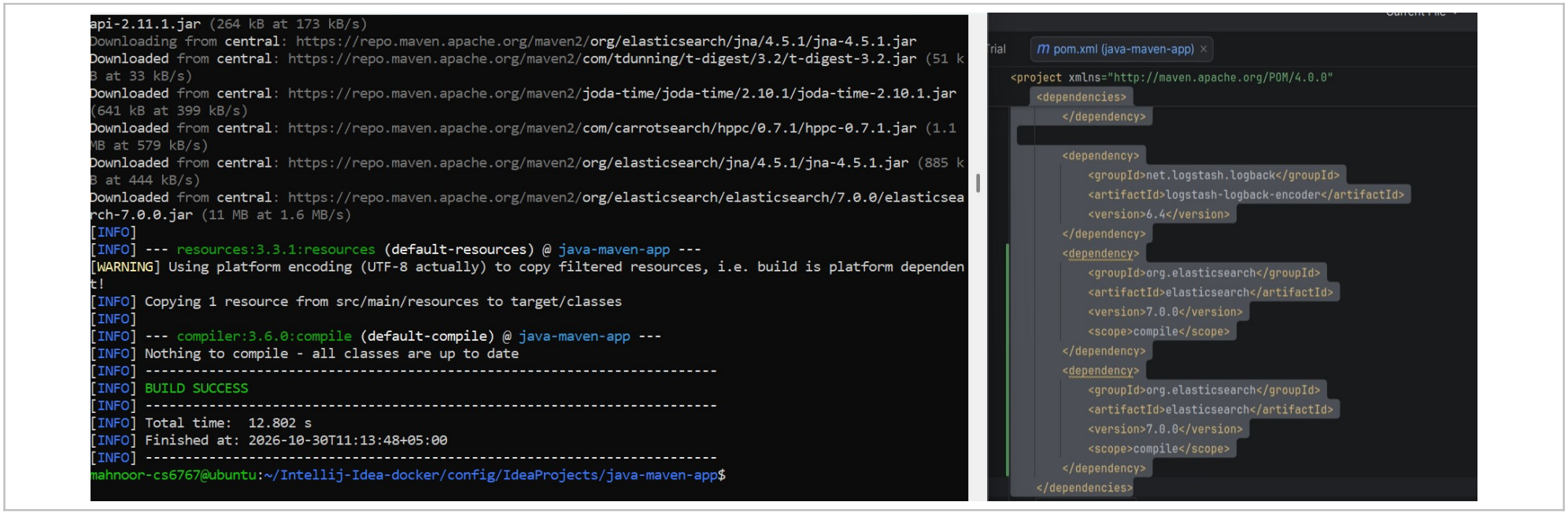


Figure 9: Elasticsearch dependency in pom.xml (right) and the Maven build that downloads it (left)

What to look for

- The terminal shows elasticsearch-7.0.0.jar (11 MB) being downloaded from Maven Central.

- The build ends with BUILD SUCCESS at 11:13:48.
- In the editor the elasticsearch block appears twice. Maven accepts this, but it likely causes the “malformed project” warning in the next step. Remove one copy for a cleaner build.

Step 7: Repackage the application

Build the JAR again, this time including the new dependency.

What to do

1. Run the package goal again:

```
mvn package
```

The command is not visible in the screenshot; this is the standard command that produces this output.

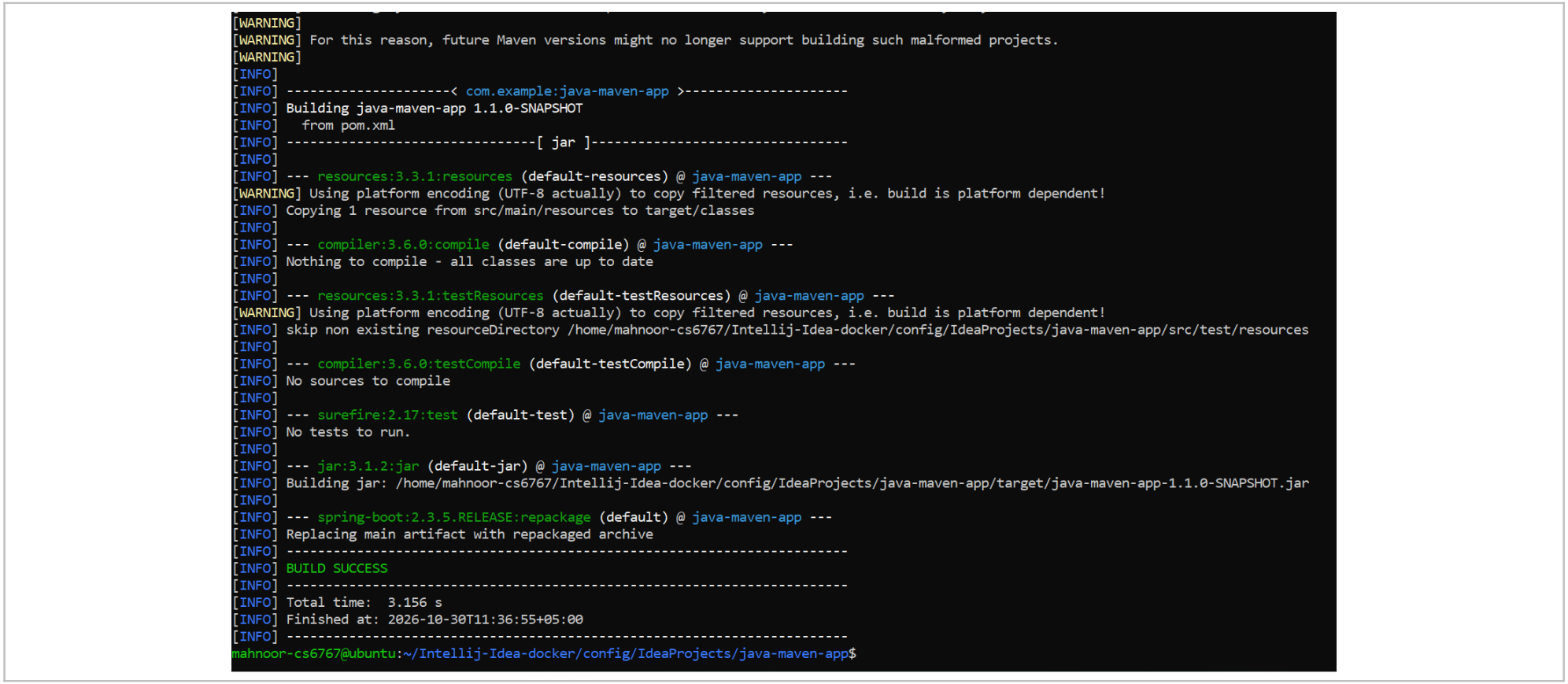


Figure 10: Maven rebuild creating and repackaging the JAR

What to look for

- The jar plugin builds java-maven-app-1.1.0-SNAPSHOT.jar.
- The spring-boot repackage step then replaces it with an executable version.

- BUILD SUCCESS in 3.156 seconds, finished at 11:36:55. The warnings near the top refer to the duplicate dependency.

Step 8: Verify the generated JAR file

Check that the build produced the JAR in the target folder.

What to do

1. List the files in the target folder:

```
ls -la /home/mahnoor-cs6767/Intellij-Idea-docker/config/IdeaProjects/java-maven-app/target/
```

```
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker/config/IdeaProjects/java-maven-app$ ls -la /home/mahnoor-cs6767/Intellij-Idea-docker/config/IdeaProjects/java-maven-app/target/
total 37840
drwxr-xr-x 6 mahnoor-cs6767 mahnoor-cs6767     4096 Oct 30 11:36 .
drwxr-xr-x 6 mahnoor-cs6767 docker             4096 Oct 30 11:13 ..
drwxr-xr-x 4 mahnoor-cs6767 mahnoor-cs6767     4096 Oct 30 10:41 classes
drwxr-xr-x 3 mahnoor-cs6767 mahnoor-cs6767     4096 Oct 30 10:41 generated-sources
-rw-r--r-- 1 mahnoor-cs6767 mahnoor-cs6767 38715576 Oct 30 11:36 java-maven-app-1.1.0-SNAPSHOT.jar
-rw-r--r-- 1 mahnoor-cs6767 mahnoor-cs6767     3071 Oct 30 11:36 java-maven-app-1.1.0-SNAPSHOT.jar.original
drwxr-xr-x 2 mahnoor-cs6767 mahnoor-cs6767     4096 Oct 30 10:41 maven-archiver
drwxr-xr-x 3 mahnoor-cs6767 mahnoor-cs6767     4096 Oct 30 10:41 maven-status
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker/config/IdeaProjects/java-maven-app$
```

Figure 11: Contents of the target folder

What to look for

- java-maven-app-1.1.0-SNAPSHOT.jar is about 38.7 MB. It is large because the repackaged JAR contains all the libraries the app needs.
- java-maven-app-1.1.0-SNAPSHOT.jar.original (3,071 bytes) is the small JAR from before repackaging.
- The folders classes, generated-sources, maven-archiver and maven-status are other build outputs.

Step 9: Run the Maven tests

Maven can run a project's automated tests with a single goal.

What to do

1. Run the test goal:

```
mvn test
```

The command is not visible in the screenshot; this is the standard command that produces this output.

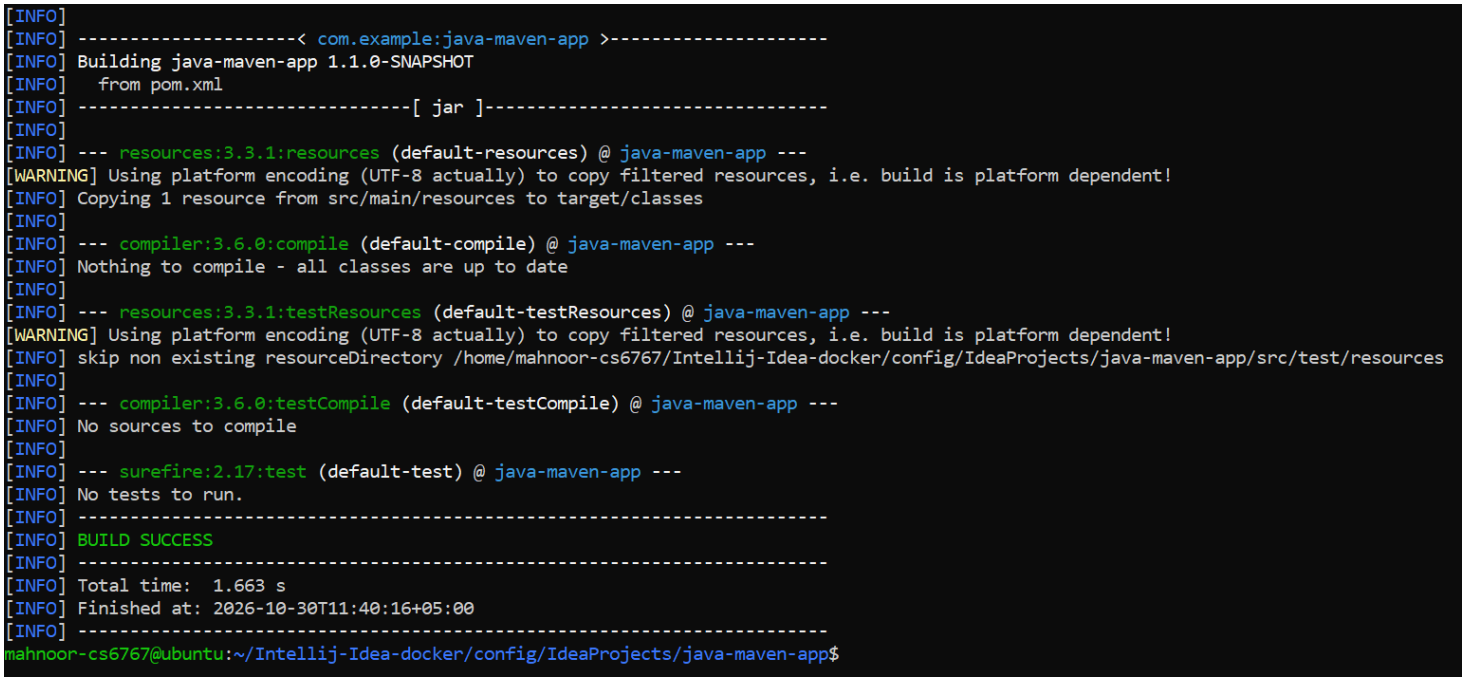


Figure 12: Maven test phase

What to look for

- "No tests to run." appears because this project has no test code ("No sources to compile" for testCompile).
- The build still ends with BUILD SUCCESS in 1.663 seconds.

Step 10: Run the application

The repackaged JAR is a complete Spring Boot application, so it can be started directly with Java.

What to do

1. Start the app:

```
java -jar target/java-maven-app-1.1.0-SNAPSHOT.jar
```

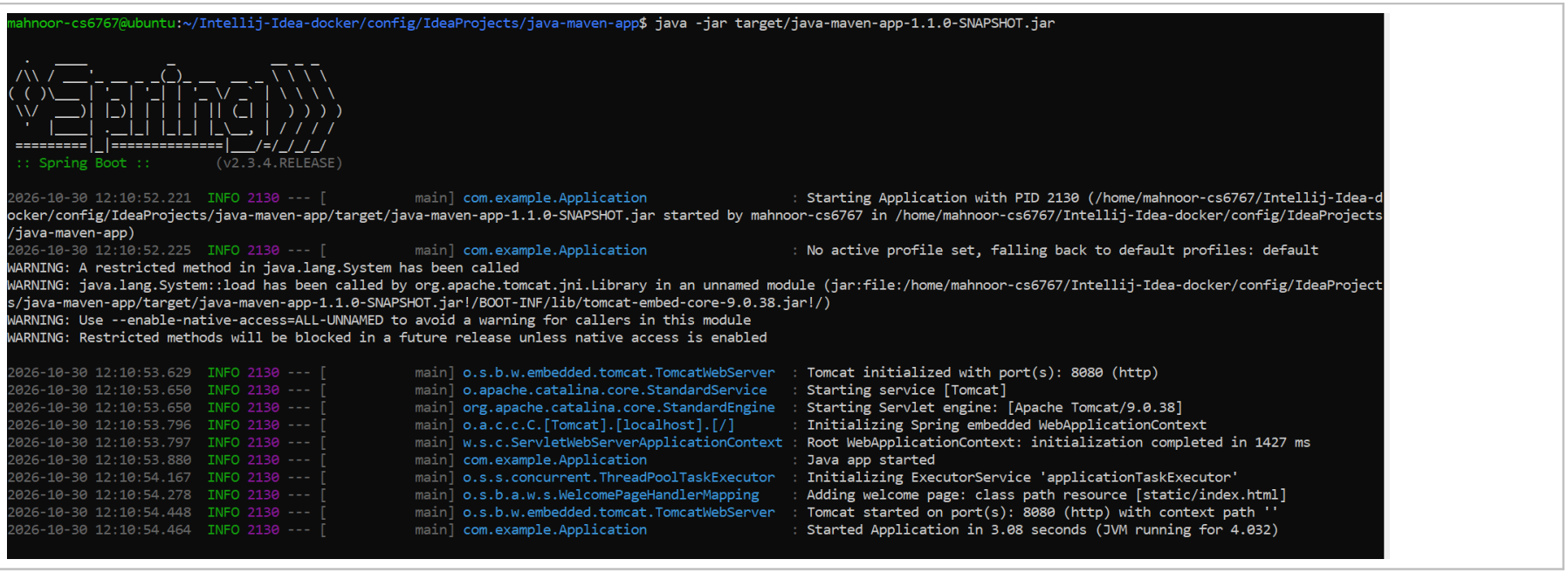


Figure 13: Spring Boot application started from the Maven JAR

What to look for

- The Spring Boot banner shows version 2.3.4.RELEASE.
- “Tomcat started on port(s): 8080 (http)” means the web server is up.
- “Started Application in 3.08 seconds” confirms a successful start. The WARNING lines about restricted methods come from the newer Java version and can be ignored.

Step 11: Stop the application and close the project

Stop the running app, then close the project in IntelliJ to go back to the Welcome screen.

What to do

1. Press Ctrl+C in the terminal to stop the app:

```
2026-10-30 12:10:54.464  INFO 2130 --- [           main] com.example.Application                  : Started Application in 3.08 seconds (JVM running for 4.032)
^C2026-10-30 12:18:03.334  INFO 2130 --- [extShutdownHook] o.s.s.concurrent.ThreadPoolTaskExecutor  : Shutting down ExecutorService 'applicationTaskExecutor'
mahnoor-cs6767@ubuntu:~/Intellij-Idea-docker/config/IdeaProjects/java-maven-app$
```

Figure 14: Application stopped with Ctrl+C

2. In IntelliJ, close the project (File > Close Project).

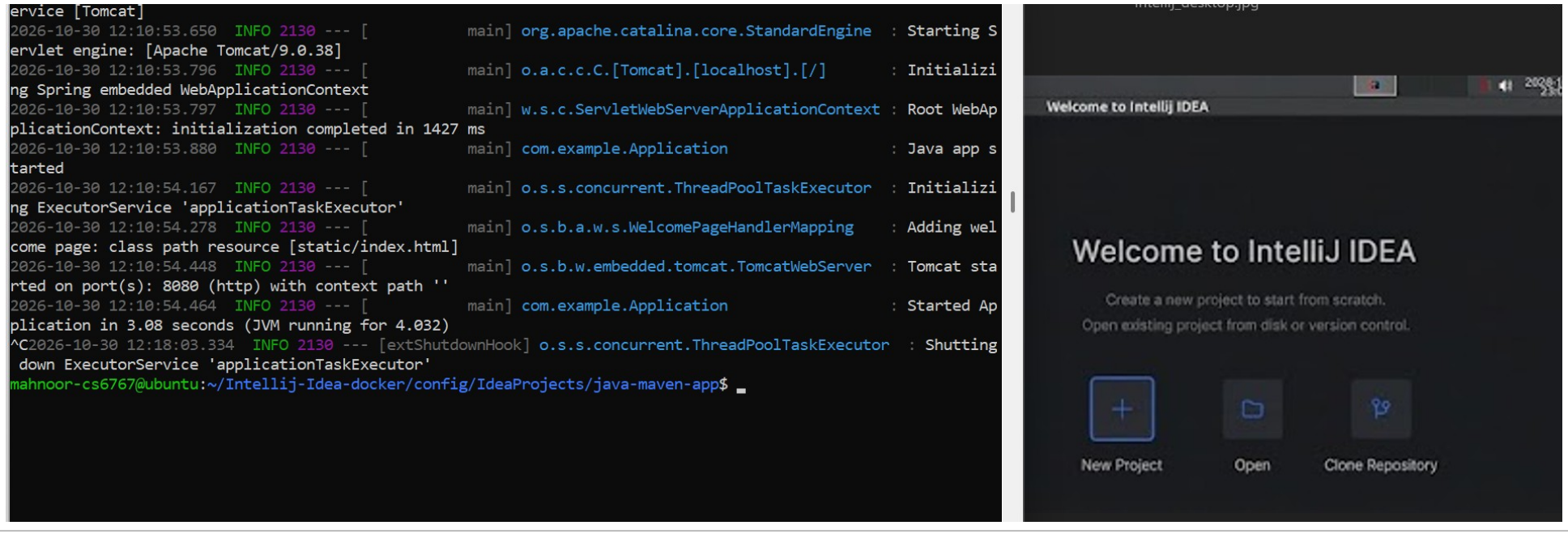


Figure 15: Application log (left) and IntelliJ Welcome screen after closing the project (right)

What to look for

- After ^C the log shows “Shutting down ExecutorService”, so the app stopped cleanly.
- IntelliJ returns to the Welcome screen.

Part 3: Build a Java Application with Gradle (java-app)

Step 12: Clone the java-app project in IntelliJ

Clone the second Java sample, which uses Gradle instead of Maven.

What to do

1. On the Welcome screen, click Clone Repository and enter:

```
URL: https://gitlab.com/nanuchi/java-app.git
Directory: /config/IdeaProjects/java-app
```

2. Click Clone.

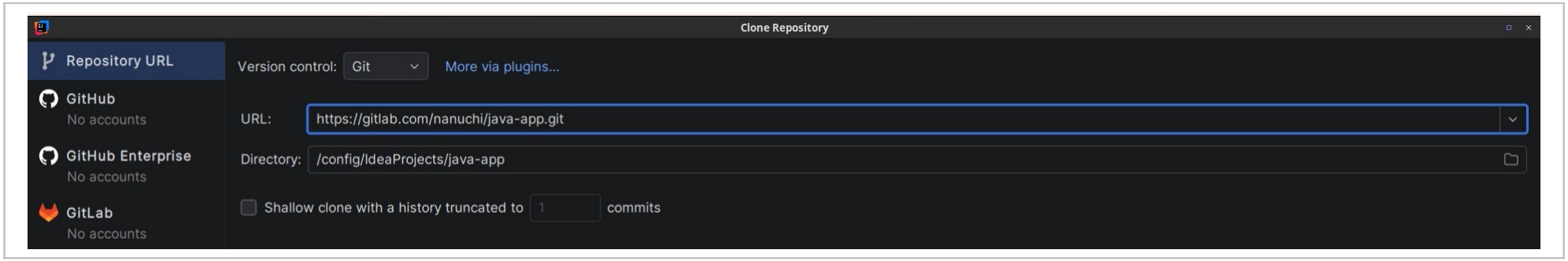


Figure 16: Clone Repository dialog for java-app

What to look for

- The URL points to nanuchi/java-app and the directory is /config/IdeaProjects/java-app.

Step 13: Build the project with Gradle

Gradle projects include a wrapper script (gradlew) that downloads the right Gradle version automatically. Here the build runs inside the IntelliJ container using Java 11.

What to do

1. Run the Gradle build inside the container:

```
docker exec -it intellij-idea-docker-intellij-1 bash -c "cd /config/IdeaProjects/java-app &&
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 ./gradlew build"
```

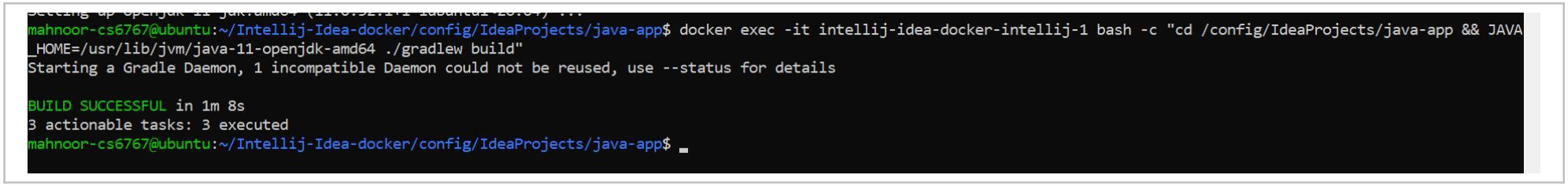


Figure 17: Gradle build finishing with BUILD SUCCESSFUL

What to look for

- BUILD SUCCESSFUL in 1m 8s, with 3 actionable tasks executed.
- The partly visible line at the top shows OpenJDK 11 being installed in the container before the build.
- The note about an incompatible Gradle daemon is informational.

Step 14: Locate the generated JAR

Gradle places its build output in the build folder.

What to do

1. In IntelliJ's Project view, expand build and then libs.

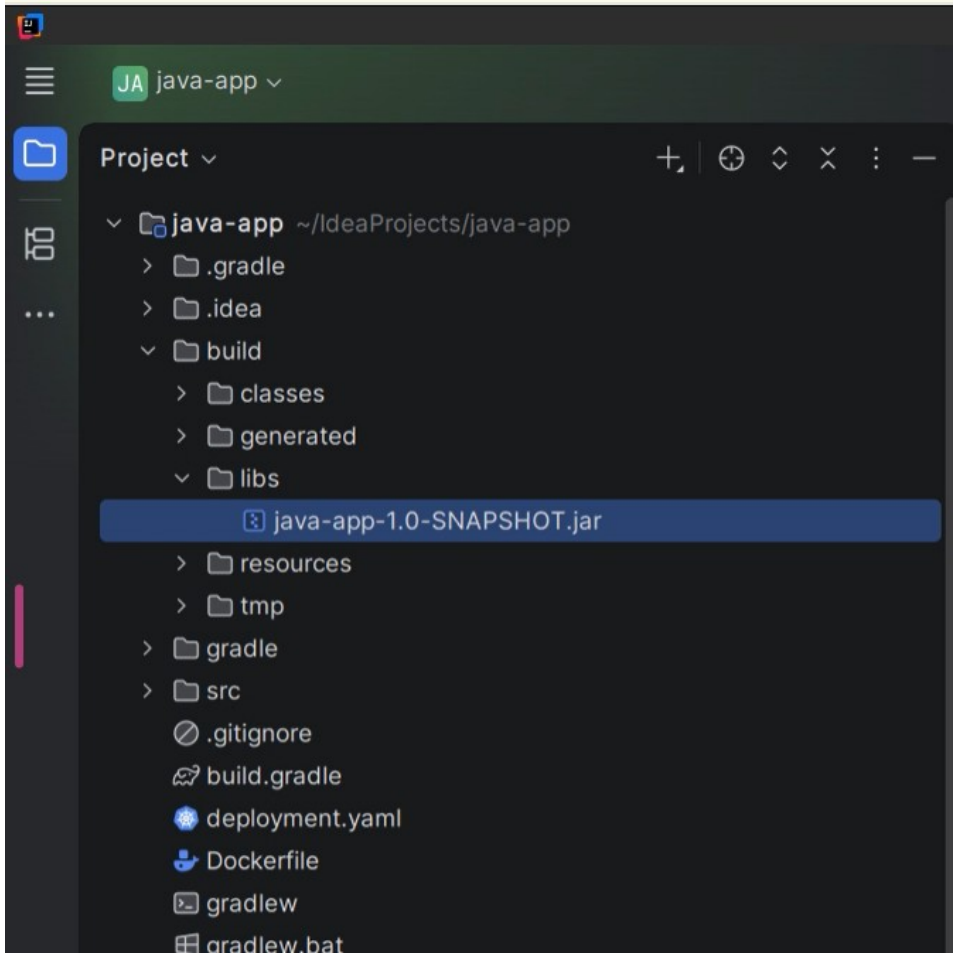


Figure 18: java-app-1.0-SNAPSHOT.jar in build/libs

What to look for

- The file java-app-1.0-SNAPSHOT.jar is in build/libs.

- The project root also contains build.gradle, Dockerfile, deployment.yaml and the gradlew scripts.

Step 15: Add the Elasticsearch dependency

In Gradle, dependencies are listed in build.gradle. Add the same Elasticsearch library that was added to the Maven project.

What to do

1. Add this line inside the dependencies block of build.gradle:

```
implementation("org.elasticsearch:elasticsearch:7.0.0")
```

2. Rebuild and force Gradle to refresh its dependencies:

```
docker exec -it intellij-idea-docker-intellij-1 bash -c "cd /config/IdeaProjects/java-app &&
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 ./gradlew build --refresh-dependencies"
```

The start of this command is cut off in the screenshot; the visible part ends with ./gradlew build --refresh-dependencies.

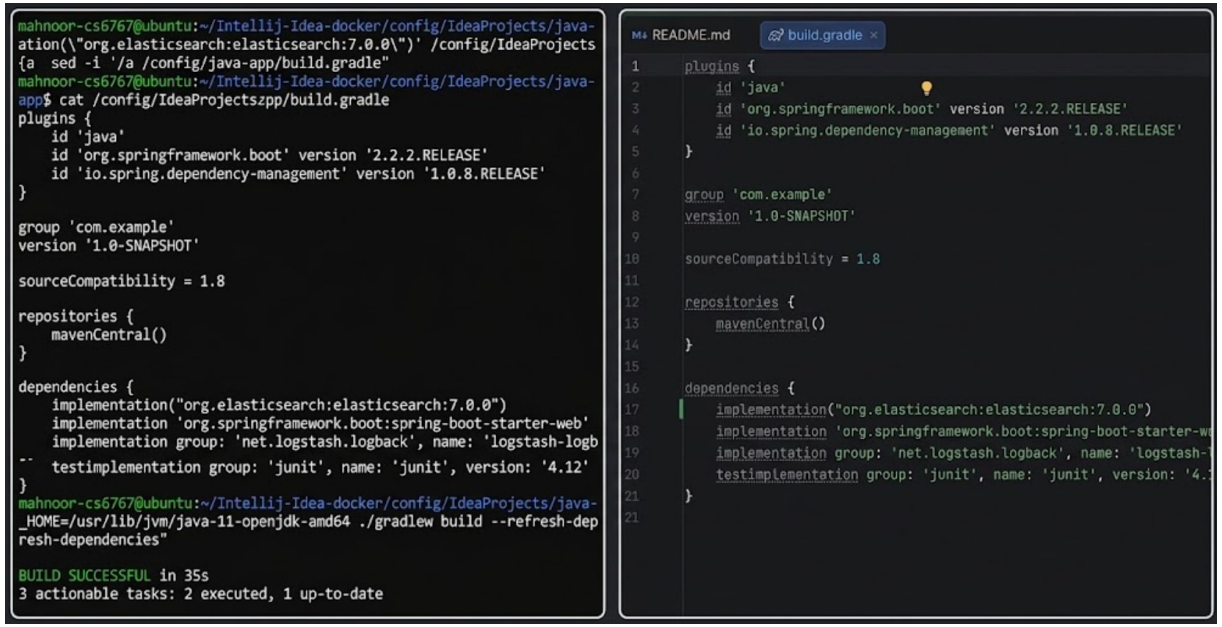


Figure 19: build.gradle with the new dependency (terminal on the left, IntelliJ editor on the right)

What to look for

- The new line is the first entry under dependencies, both in the terminal listing (cat) and at line 17 in the editor.
- BUILD SUCCESSFUL in 35s with 2 tasks executed and 1 up-to-date.

Step 16: Rebuild the project without tests

Rebuild the JAR so it contains the new dependency. This time the tests are skipped with -x test.

What to do

1. Make sure the container is running:

```
docker start intellij-idea-docker-intellij-1
```

2. Build without the daemon and without tests:

```
docker exec -it intellij-idea-docker-intellij-1 bash -c "cd /config/IdeaProjects/java-app &&
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 ./gradlew --no-daemon build -x test"
```



Figure 20: Gradle rebuild with --no-daemon and -x test

What to look for

- BUILD SUCCESSFUL in 1m 46s; all 3 tasks are up to date because nothing had changed since the last build.
- The message about a single-use daemon appears because of the --no-daemon option.

Step 17: Run the application

Start the Gradle-built JAR inside the container.

What to do

1. Run the JAR:

```
docker exec -it intellij-idea-docker-intellij-1 bash -c "cd /config/IdeaProjects/java-app &&
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 java -jar build/libs/java-app-1.0-SNAPSHOT.jar"
```

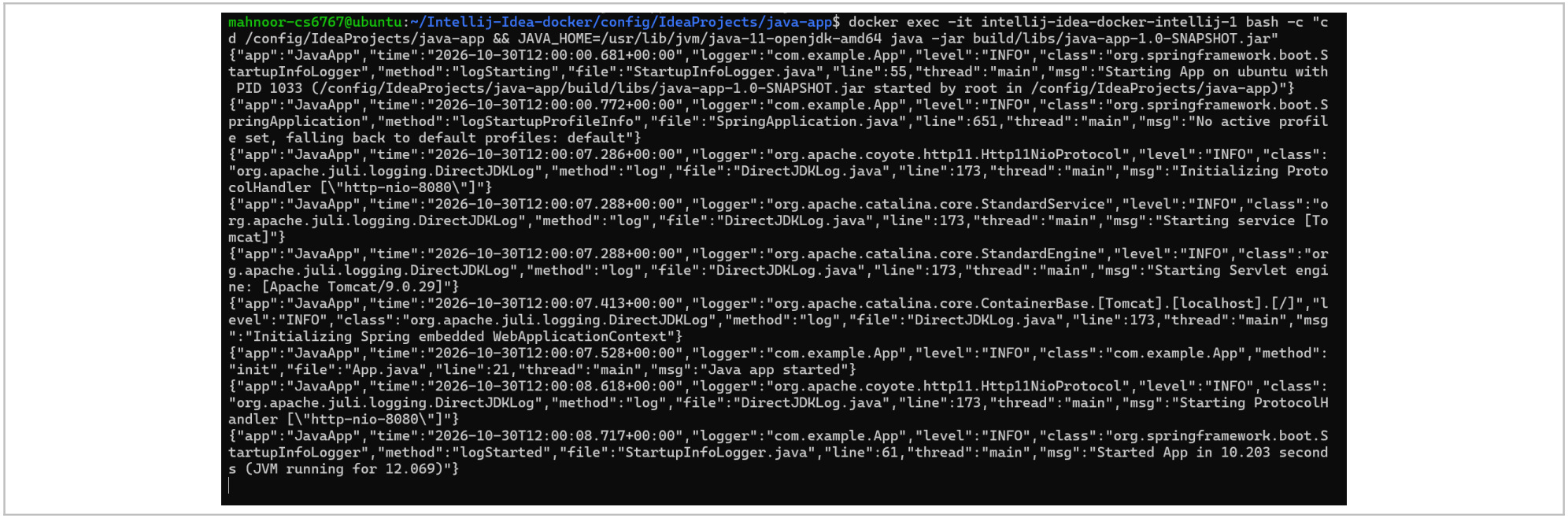


Figure 21: java-app running with JSON-formatted logs

What to look for

- The logs are printed as JSON lines.
- Tomcat 9.0.29 initializes on port 8080 (http-nio-8080).
- “Started App in 10.203 seconds” confirms that the app is running.

Step 18: Stop the application and close the project

Stop the app and return to the IntelliJ Welcome screen.

What to do

1. Press Ctrl+C in the terminal:



Figure 22: Application stopped with Ctrl+C

2. Close the project in IntelliJ.



Figure 23: IntelliJ Welcome screen listing both projects

What to look for

- The prompt returns after ^C, so the app has stopped.
- The Welcome screen now lists both java-app and java-maven-app under Projects.

Part 4: Build and Package a Node.js Application with npm (node-project)

Step 19: Clone the node-project

Clone the Node.js sample application.

What to do

1. On the Welcome screen, click Clone Repository and enter:

```
URL: https://gitlab.com/devopsbootcamp3/node-project.git
Directory: /config/IdeaProjects/node-project
```

2. Click Clone.

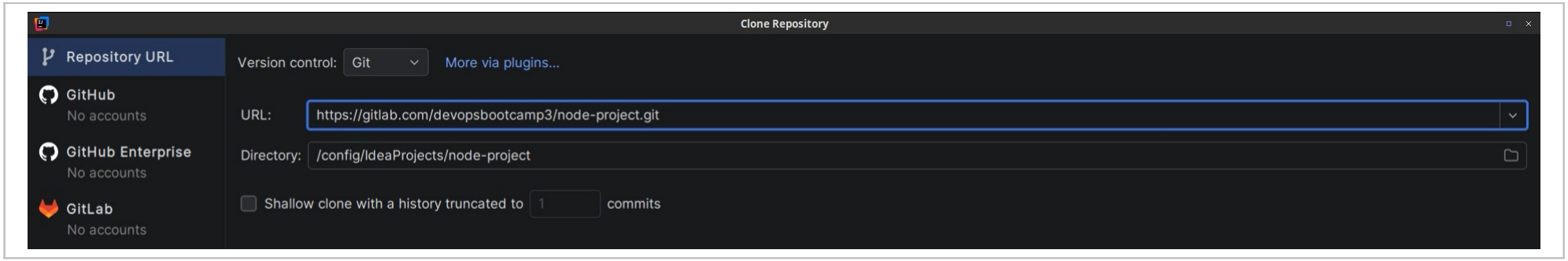


Figure 24: Clone Repository dialog for node-project

What to look for

- The URL points to devopsbootcamp3/node-project and the directory is /config/IdeaProjects/node-project.

Step 20: Install the project dependencies

npm reads package.json and downloads every library the application needs into node_modules.

What to do

1. Run:

```
npm install
```

The command is not visible in the screenshot; this is the standard command that produces this output.

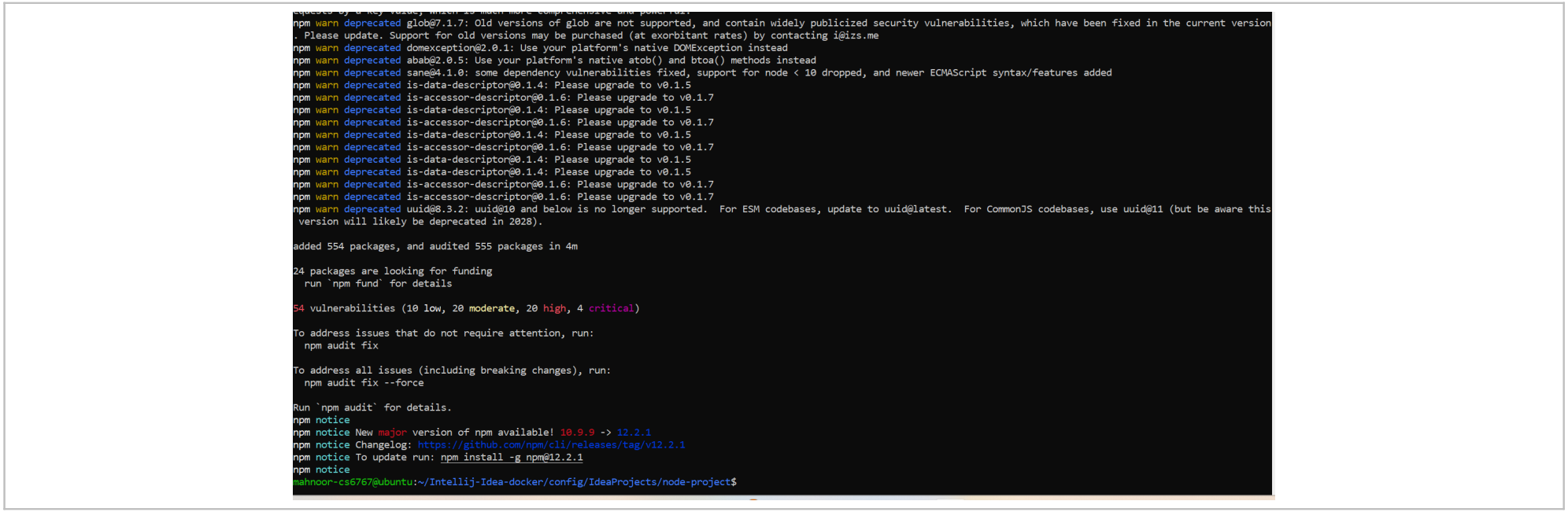


Figure 25: npm install adding 554 packages

What to look for

- "added 554 packages, and audited 555 packages in 4m" means the install finished.
- The "deprecated" warnings are normal for an older project.
- npm audit reports 54 known vulnerabilities (10 low, 20 moderate, 20 high, 4 critical) in these older libraries. This does not stop the install, and npm audit fix would address some of them.

Step 21: Run the automated tests

The project includes a Jest test. Run it from the app folder using IntelliJ's terminal.

What to do

1. Run:

```
npm test
```

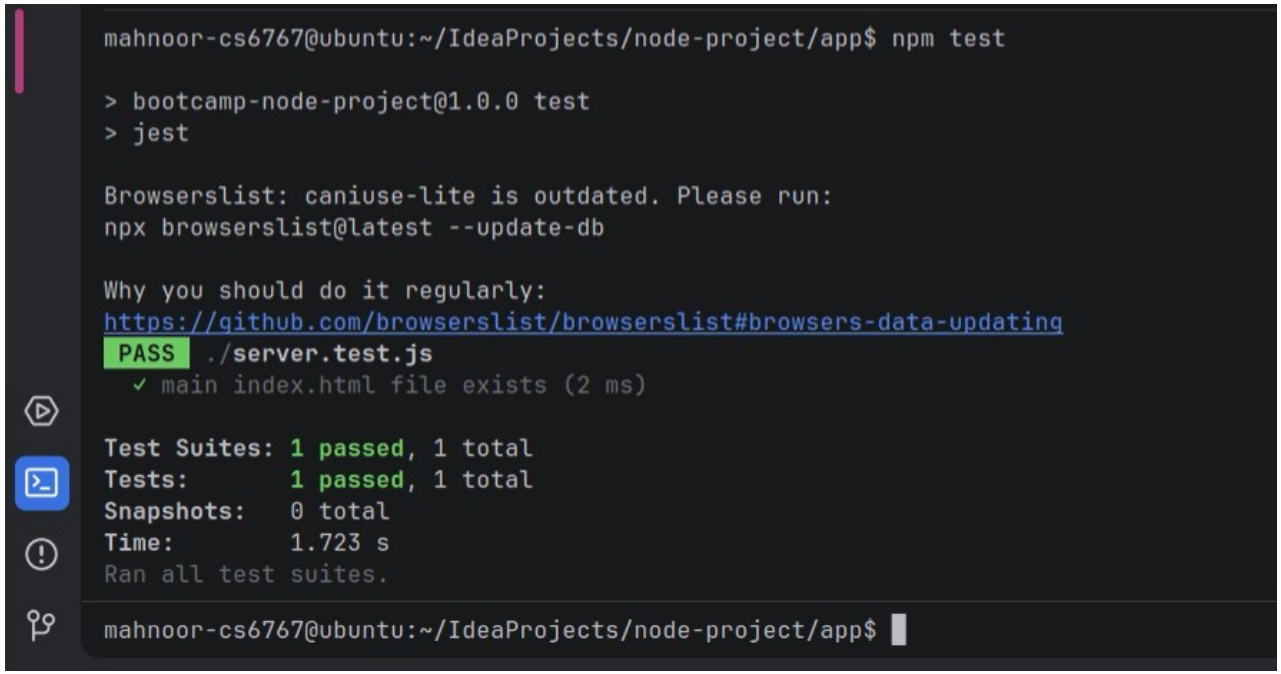


Figure 26: Jest test passing

What to look for

- npm starts jest, which runs ./server.test.js.
- The test "main index.html file exists" passes (PASS).
- The summary shows 1 test suite and 1 test passed. The Browserslist message is only a reminder to update a data file.

Step 22: Package the application with npm pack

npm pack creates a compressed .tgz archive of the project, which is how Node.js packages are shared. First the folder permissions are opened so the build can write its output.

What to do

1. Give all users access to the project folder:

```
sudo chmod -R 777 /config/IdeaProjects/node-project
```

777 is very permissive. It is acceptable for a lab environment but should not be used in production.

2. Create the package:

```
npm pack
```

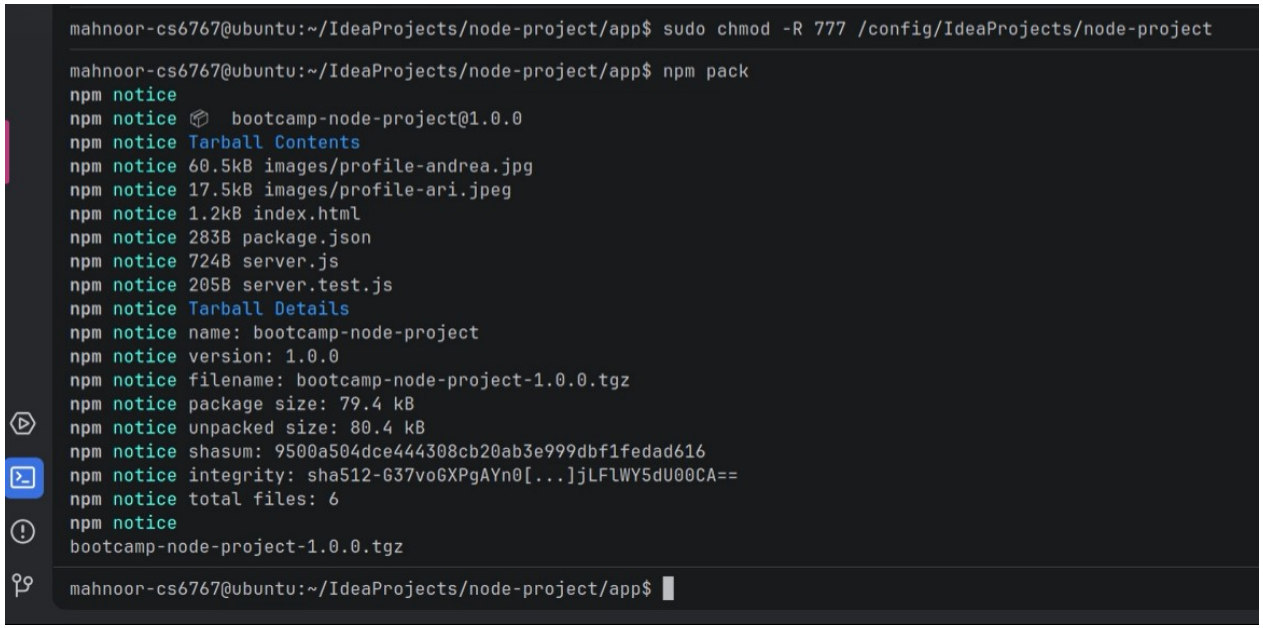


Figure 27: npm pack creating the .tgz package

What to look for

- The tarball contains 6 files: two profile images, index.html, package.json, server.js and server.test.js.
- The output file is bootcamp-node-project-1.0.0.tgz (package size 79.4 kB, unpacked size 80.4 kB).

Step 23: Run the application

Start the Node.js server inside the container.

What to do

1. Run:

```
docker exec -it intellij-idea-docker-intellij-1 bash -c "cd /config/IdeaProjects/node-project/app && npm start"
```

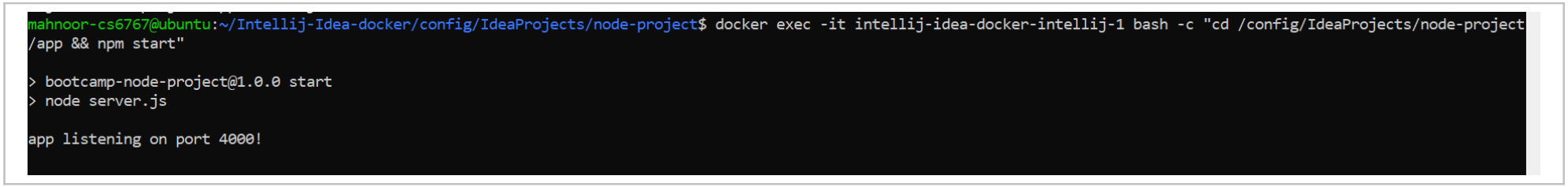


Figure 28: Node.js app listening on port 4000

What to look for

- npm start runs node server.js.
- The app prints “app listening on port 4000!”.

Step 24: Open the application in a browser

Check the running app in a web browser.

What to do

1. Open http://localhost:4000 in a browser tab.

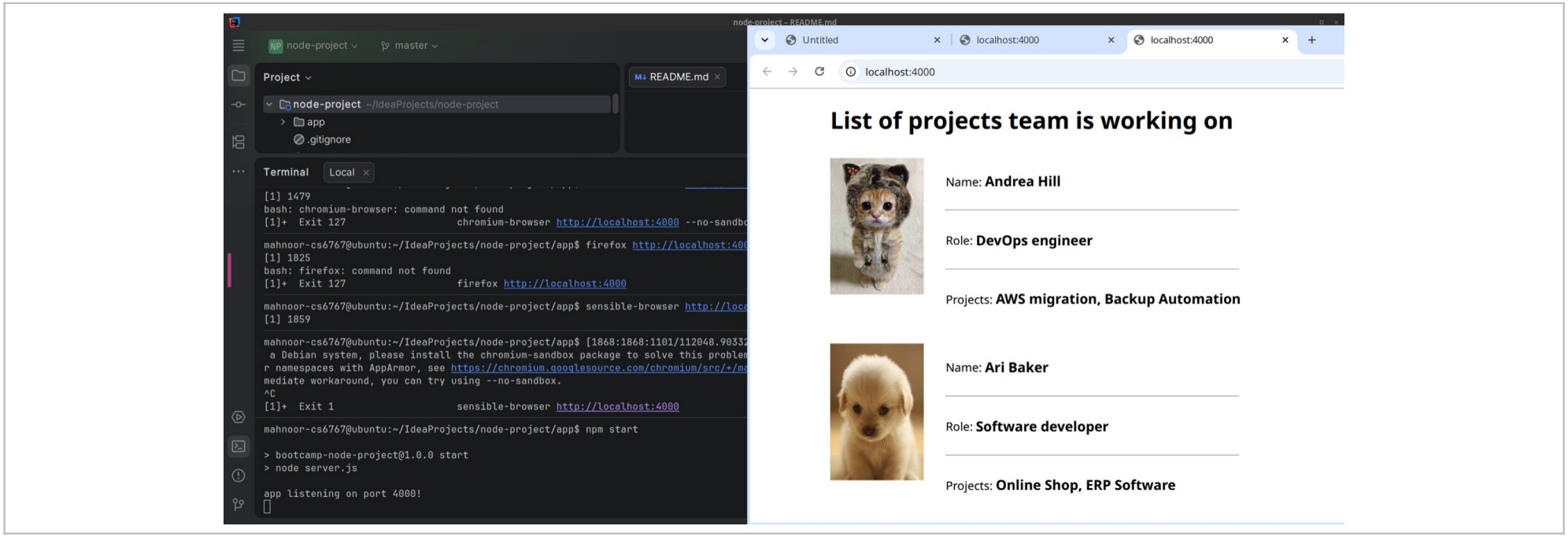


Figure 29: Node.js application in the browser (right) with the IntelliJ terminal (left)

What to look for

- The page “List of projects team is working on” shows two team members: Andrea Hill (DevOps engineer) and Ari Baker (Software developer), each with their projects.
- The terminal on the left shows earlier attempts to open chromium-browser and firefox from the terminal. They failed with “command not found” because those browsers are not installed there. The page was opened in a normal browser tab instead.
- npm start shows “app listening on port 4000!” at the bottom.

Part 5: Build a Docker Image for the Node.js Application

Step 25: Check that the Dockerfile exists

A Docker image is built from a Dockerfile. Make sure the project contains one.

What to do

1. In the node-project folder, list the files:

```
ls
```

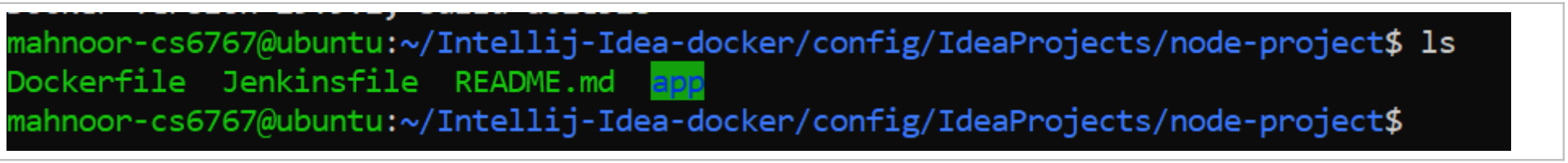


Figure 30: node-project folder contents

What to look for

- The folder contains Dockerfile, Jenkinsfile, README.md and the app folder.

Step 26: List the Docker images before building

Check which images exist now, so you can see the new one appear after the build.

What to do

1. Run (sudo asks for your password):

```
sudo docker images
```

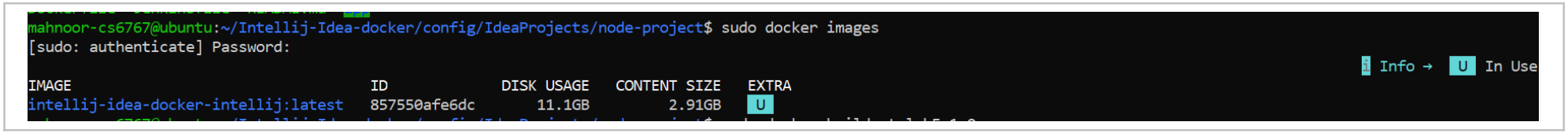


Figure 31: Docker images before the build

What to look for

- Only one image is listed: intellij-idea-docker-intellij:latest (about 11.1 GB disk usage).

Step 27: Build the Docker image

docker build reads the Dockerfile and creates an image. The -t option gives the image a name and a tag, and the final dot means “use the current folder”.

What to do

1. Build the image and name it lab5:1.0:

```
sudo docker build -t lab5:1.0 .
```

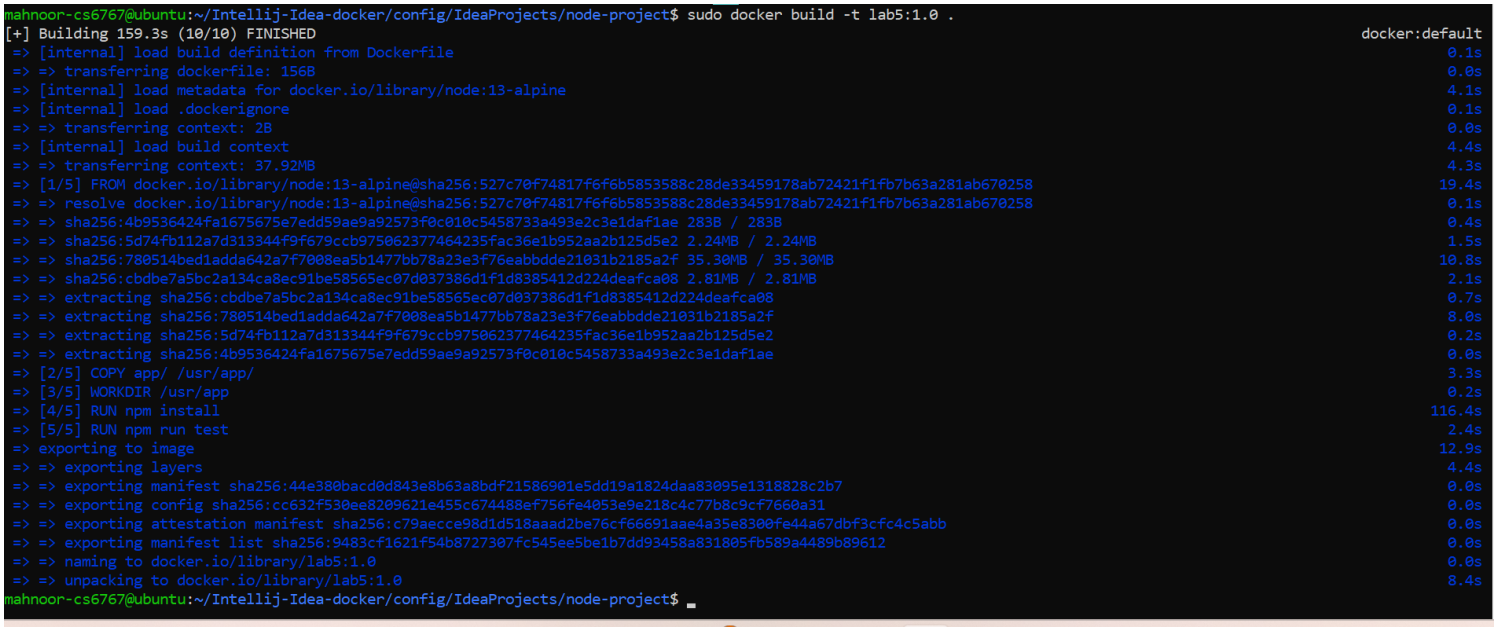


Figure 32: Docker build output for lab5:1.0

What to look for

- The base image is node:13-alpine.
- The five build stages are: FROM node:13-alpine, COPY app/ /usr/app/, WORKDIR /usr/app, RUN npm install and RUN npm run test.
- The build finishes (10/10) in 159.3 seconds, and the last lines name the image docker.io/library/lab5:1.0.

Step 28: Verify the new image

List the images again to confirm that the new image exists.

What to do

1. Run:

```
sudo docker images
```

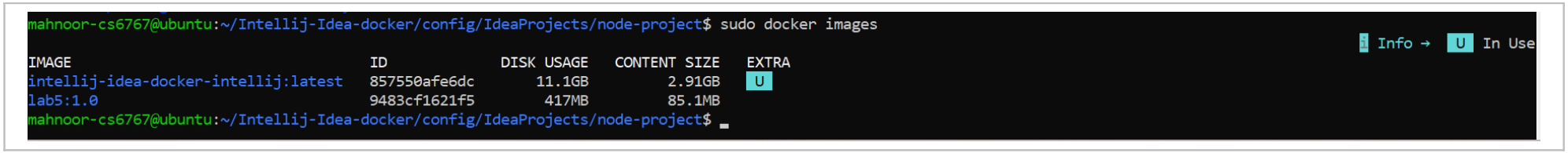


Figure 33: Docker images after the build

What to look for

- lab5:1.0 now appears with image ID 9483cf1621f5.
- Its size is 417 MB on disk (85.1 MB of content).

Part 6: Build the API of a React and Node.js Project (react-nodejs-example)

Step 29: Clone the react-nodejs-example project

Clone a project that has a React front end and a Node.js API.

What to do

1. On the Welcome screen, click Clone Repository and enter:

```
URL: https://gitlab.com/bbachi/react-nodejs-example.git
Directory: /config/IdeaProjects/react-nodejs-example
```

2. Click Clone.



Figure 34: Clone Repository dialog for react-nodejs-example

What to look for

- The URL points to bbachi/react-nodejs-example and the directory is /config/IdeaProjects/react-nodejs-example.

Step 30: Install the API dependencies

The Node.js API lives in the api folder. Install its dependencies there.

What to do

1. In the api folder, run:

```
npm install
```

The command is not visible in the screenshot; this is the standard command that produces this output.

2. Check the folder contents and the first few installed packages:

```
ls -la
ls node_modules | head -n 10
```

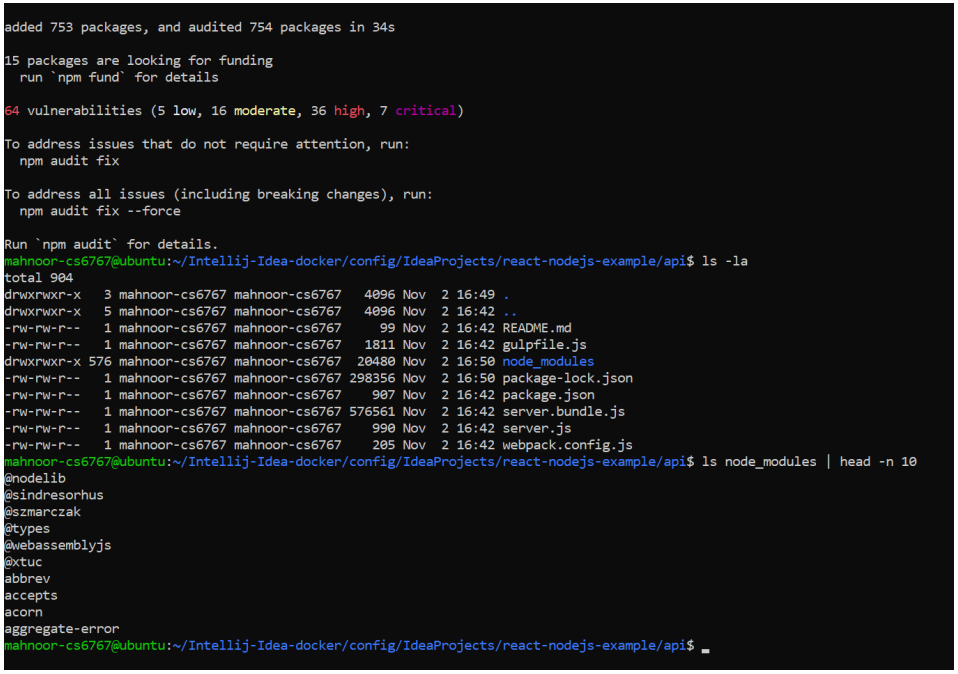


Figure 35: API dependencies installed in the api folder

What to look for

- “added 753 packages, and audited 754 packages in 34s” confirms the install.
- npm audit again reports vulnerabilities (64 in total, including 7 critical) in these older libraries.
- The api folder now has node_modules and package-lock.json next to the project files (README.md, gulpfile.js, package.json, server.js, webpack.config.js).

- ls node_modules lists the first installed packages, such as @nodelib, @types, abbrev and accepts.

Step 31: Build the server bundle with webpack

webpack combines the server code and its libraries into one file, server.bundle.js. The build script is run with an extra Node.js option, because this older version of webpack needs it to work with a newer Node.js.

What to do

1. Run the build and then check the output file:

```
NODE_OPTIONS=--openssl-legacy-provider npm run build
ls -la server.bundle.js
```

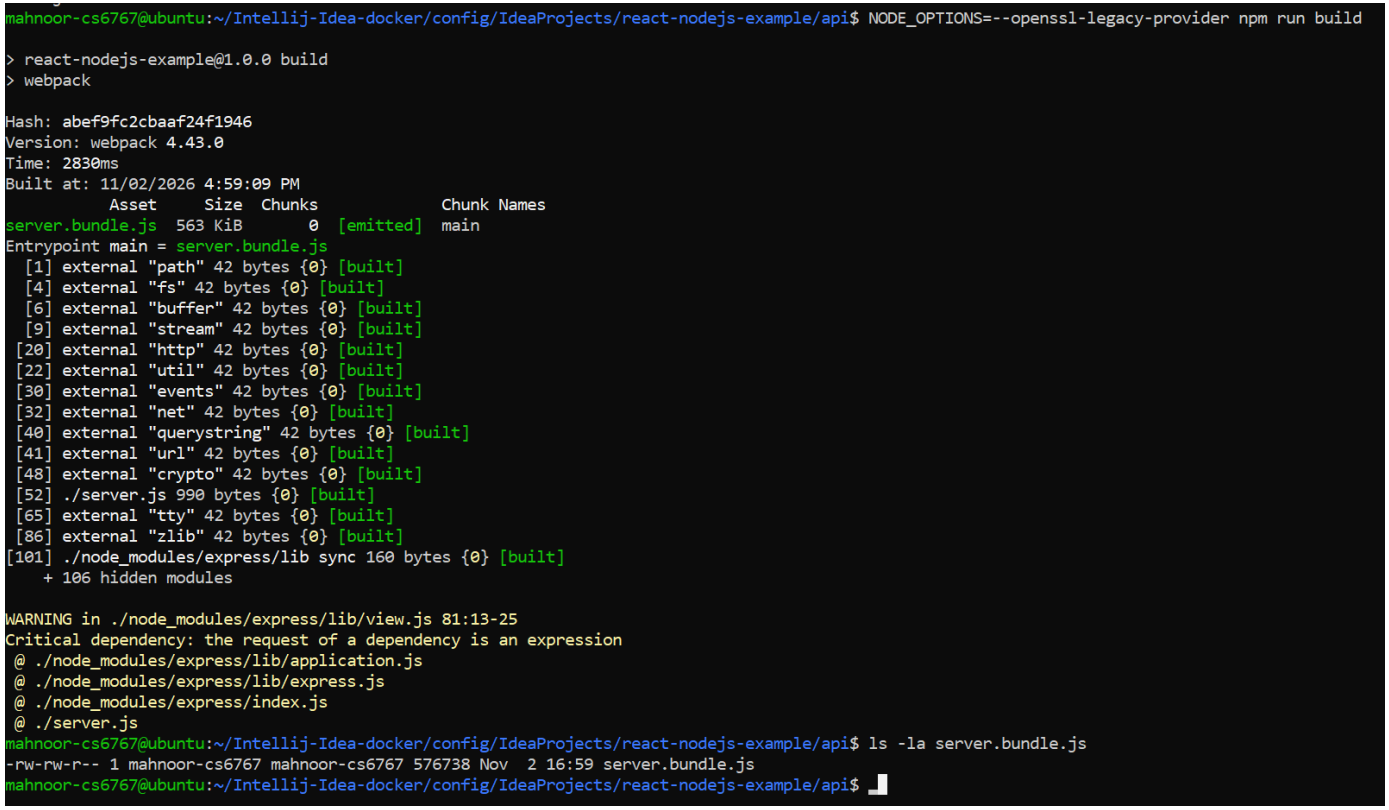


Figure 36: webpack build and the generated server.bundle.js

What to look for

- webpack 4.43.0 finishes in 2830 ms and emits server.bundle.js (563 KiB).
- The “Critical dependency” warning comes from the Express library and does not stop the build.
- ls shows server.bundle.js (576,738 bytes) with a new timestamp of 16:59.

Step 32: Inspect the bundle

Look inside the generated file to see what webpack produced.

What to do

1. Print the file page by page:

```
cat server.bundle.js | more
```

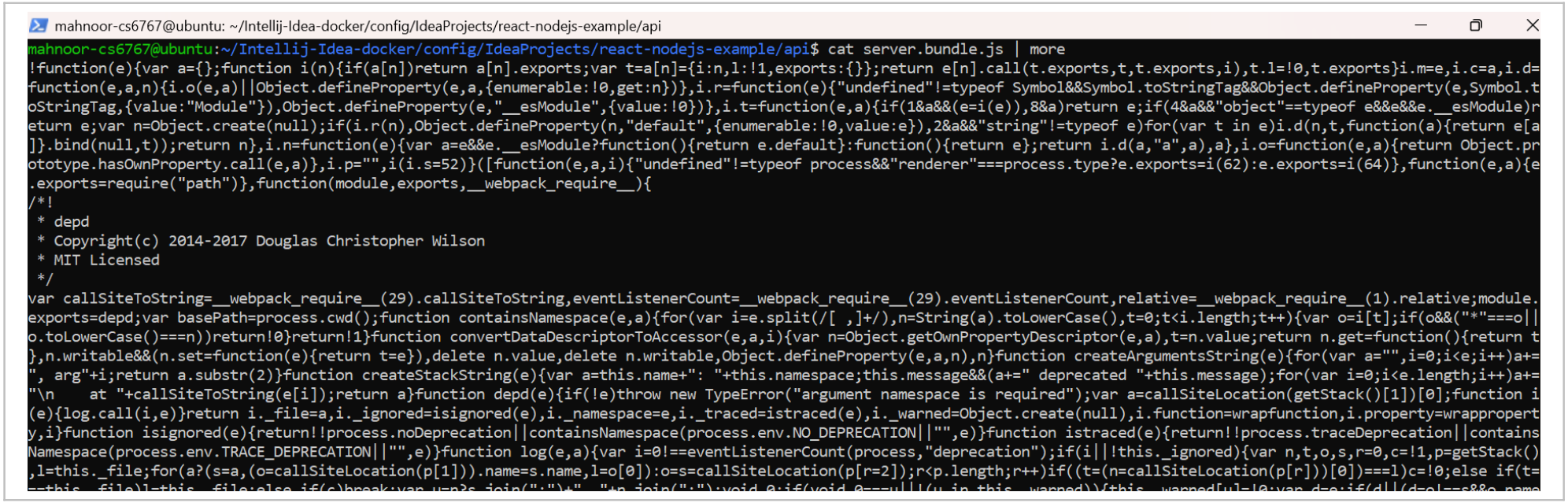


Figure 37: Start of server.bundle.js

What to look for

- The file contains minified (compressed) JavaScript, so it is hard to read but fast to load.
- It includes the code of the libraries used by the server, such as the depd package whose license header is visible.

Summary of Build Results

Project	Tool	Output	Result
java-maven-app	Maven 3.9.12	target/java-maven-app-1.1.0-SNAPSHOT.jar (about 38.7 MB)	BUILD SUCCESS
java-app	Gradle (wrapper)	build/libs/java-app-1.0-SNAPSHOT.jar	BUILD SUCCESSFUL
node-project	npm	bootcamp-node-project-1.0.0.tgz (79.4 kB)	1 test passed
node-project	Docker	Image lab5:1.0 (417 MB)	Built in 159.3 s
react-nodejs-example/api	webpack (via npm)	server.bundle.js (563 KiB)	Build completed

Conclusion

In this lab, you set up IntelliJ IDEA in a Docker container and used it to build four projects. With Maven and Gradle you compiled Java applications, added the Elasticsearch dependency, produced runnable JAR files and started them with Spring Boot. With npm you installed dependencies, ran a Jest test, created an npm package and ran a Node.js web application. You then packaged the Node.js application as a Docker image (lab5:1.0), and finally built the API of a React and Node.js project into a single webpack bundle. Together these steps show how build tools turn source code into deployable artifacts.